

CSE 599S Homework 1 Report

Darby Brillon, Vattanary Tevy, Arnav Mazumder

May 1, 2026

0 Training Infrastructure Setup

0.1 Sanity Checks

When developing our training infrastructure, we did not make any changes to the original codebase. Our tokenizer is whitespace-centric; it splits text by whitespace and assigns tokens to each unique sequence of characters. We use the provided nanoGPT transformer implementation and API for our models.

Our training infrastructure supports three types of token-based masking when computing loss: mask the first n tokens, not masking any tokens, and mask all tokens except the one after a marker (for modular operation data, this marker is “=”). The first two aforementioned masks are used for our sanity checks, while the third is used for our experiments. We do not use any padding; since we tokenize on whitespace, every modular equation will have the same number of tokens. Our sanity checks use a one string dataset, so we don’t need padding for it either.

Our training API supports several hyperparameters, which are mentioned in our experiments section. Our API has the option to checkpoint the model halfway through, will checkpoint the model at the end, write metrics to a json file, and also return both model and metrics. For modular equation datasets that consist of multiple operations, our metrics will return along with total loss and accuracy per epoch, the loss and accuracy per epoch of each operation.

For our sanity checks, the nanoGPT model hyperparameters we used were batch size of 1, block size of 16, 1 layer, 1 head, embedding dimension of 64, dropout of 0, with bias, a learning rate of 10^{-2} , and 1000 epochs. For our dataset, we used the single string “I love machine learning”. Our first sanity check trained the model with loss computed on every token (no mask). The model was then successfully able to regurgitate the string from the empty string. For our second sanity check, we masked the first 3 tokens and computed loss on the remaining tokens, after which the model was able to complete the string given the first 3 tokens. The code for our sanity checks can be found in `sanity_checks.py` and our model checkpoints and training logs can be found in the folder `sanity_checks`.

1 Training on Algorithmic Tasks

1.1 Data Generation

For our data generation process for modular operations, including addition, subtraction, and division, we enumerate all possible equations. Our datasets are `.txt` files consisting of equations, each separated by newline characters. An equation is of the form $\mathbf{a} \circ \mathbf{b} = \mathbf{c}$, where a, b, c are integers, and $\circ$ is one of the three mentioned modular operations. For modular addition and subtraction, the possible range of a and b is $0 \leq a, b < p$. So, for addition and subtraction, the total number of equations is p^2 each (number of ordered pairs a, b , since they determine c). For division, we have that $0 \leq a < p$ and $0 < b < p$, so there are exactly

$p(p-1)$ ordered pairs a, b .

For addition and subtraction experiments, we first do an 80-20 split between data used to develop the model and that used to test it. With that 80% of the data, we do another 80-20 split between actual training data and validation data. So the final split comes out to be 64% training, 16% validation, and 20% test. For $p = 97$ ($2 \cdot 97^2 = 18818$ data points), this is 12043 training points, 3010 validation points, and 3765 test points. For $p = 113$ ($2 \cdot 113^2 = 25538$ data points), this is 16344 training points, 4086 validation points, and 5108 test points.

For division experiments, we only use $p = 97$ ($97 \cdot 96 = 9312$ data points). Our initial division experiment to get the model to Grok involves a 19 – 81 train-test split. We do not employ a validation set here because we do not do hyperparameter tuning; our hyperparameters are exactly those found in the paper for this experiment. This amounts to 1769 training points and 7543 test points.

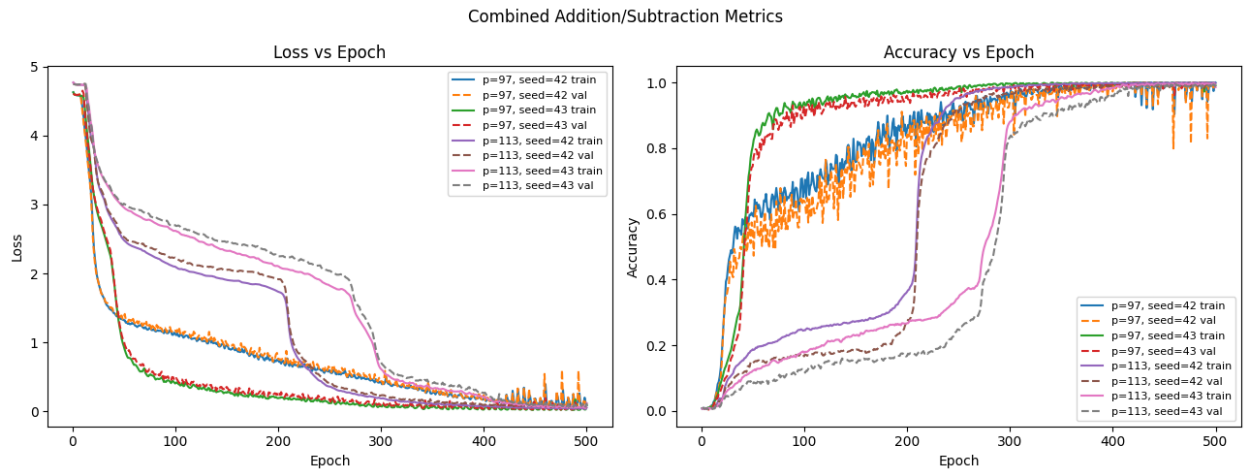
The data files can be found in each experiment’s folder. More details about their location and contents can be found in the experiments sections.

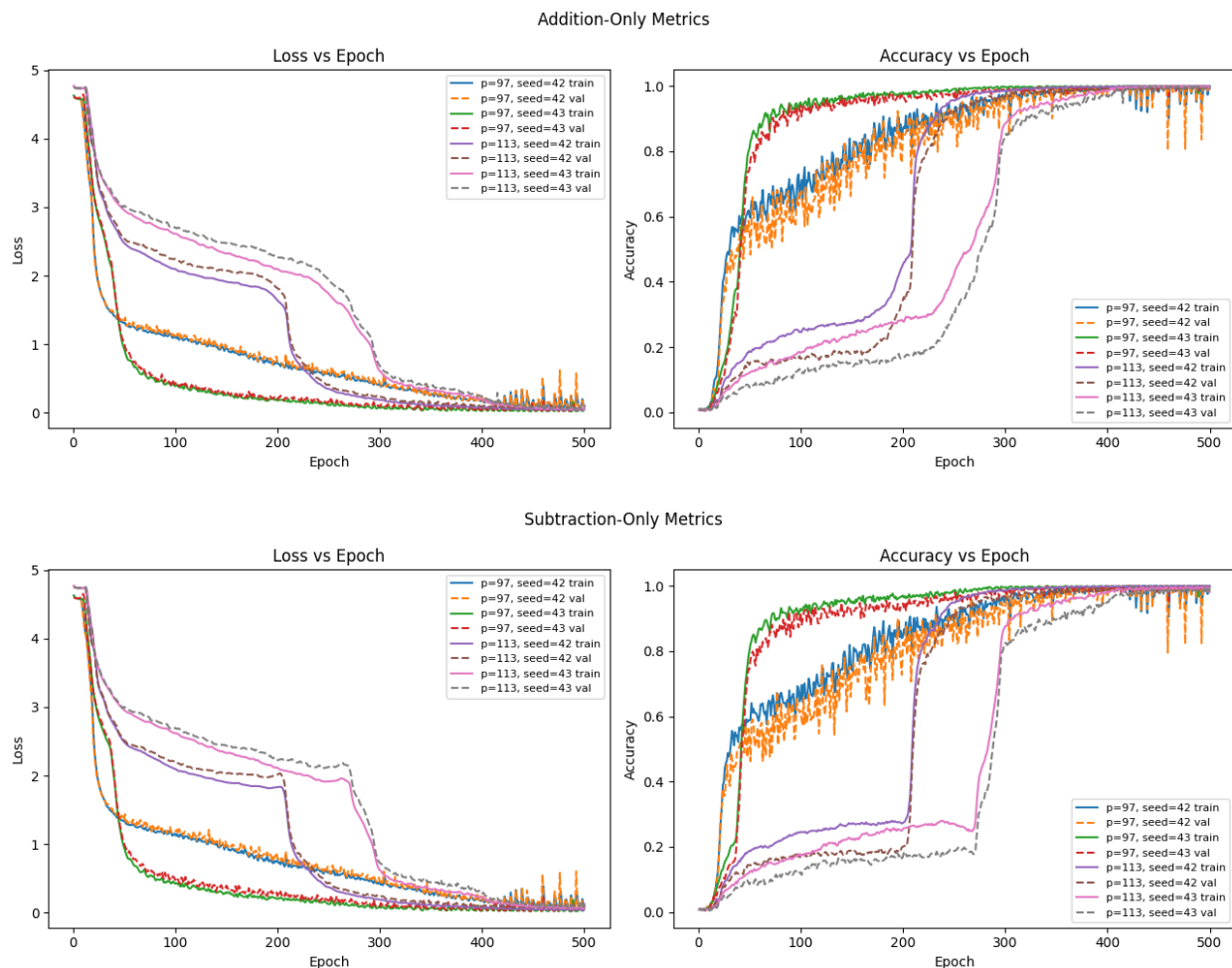
1.2 Addition and Subtraction Experiments

For our addition and subtraction experiments, we generate data and train models for $p = 97, 113$ over 2 random seeds each (4 models total). After performing hyperparameter tuning via our validation sets, we chose the following parameters for our final nanogpt models: batch size of 512, block size of 16, 2 layers, 4 heads, embedding dimension of 128, 0 dropout, with bias, learning rate of 1e-3, 500 epochs, weight decay of 1, β_1 of 0.9, β_2 of 0.92, and 10 learning rate warm-up steps.

You can find the data split files, model checkpoints, and metrics for each (p, seed) pair in the `add_subtract` folder. We used `train_plot_add_subtract.ipynb` to train our models and obtain our plots. When the `add_subtract` directory exists, this notebook will simply reproduce the plots without retraining. We’ve also provided an inference file `infer_add_subtract.py` that writes test set examples to files in the `add_subtract` directory for each model. You can modify this file to run inference on a particular model using the `write_test_outputs` function that takes a model run (defined at the top of the file), a path to your test `txt` file, and a path to write the outputs.

Below are our learning curves for all tested models for addition, subtraction, and combined. All models performed with $> 98\%$ accuracy on the test set (not used for training or hyperparameter tuning). We also observe minor “Grokking” for $p = 113$ in this regime, indicating that the problem complexity may amplify the phenomenon on a fixed training set.





1.3 Grokking

To be able to get our model to reliably grok, we referred to the grokking paper and used the same model configuration and hyperparameters as they did. Specifically, we fixed our nanoGPT model architecture and hyperparameters to

- Model: nanoGPT (2 layers, 4 heads, embedding size of 128)
- Block size of 16, batch size of 512
- Dropout of 0.0, bias: True
- Optimizer: AdamW, $\beta_1 = 0.9$, $\beta_2 = 0.98$.
- Learning weight of 10^{-3} , weight decay of 1.0
- Epochs: 2000, warmup: 10 steps

These parameters led to a model size of 0.41M parameters as which is confirmed in the grokking paper. As we used the model architecture and hyperparameters from the grokking paper we did not utilize a validation set for hyperparameter tuning.

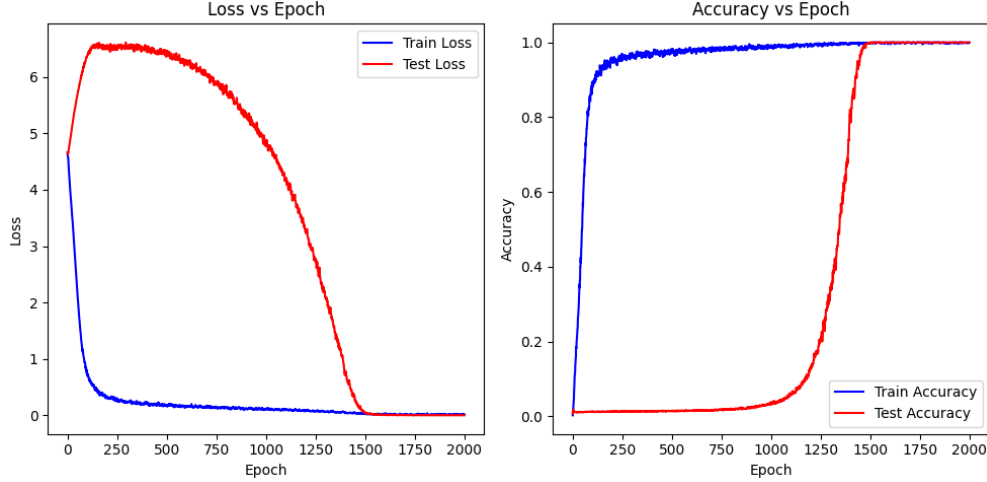


Figure 1: Demonstration of nanoGPT Model Grokking; $p = 97$, Division Operator

We trained our model on the modulo division task for a value of $p = 97$. During training we realized that the smaller the fraction of data used for training was the longer the model took to grok and if it was too large the model failed to grok. As a result we decided on using 19% of total data as our training data. This allowed our model to grok in a reasonable amount of epochs while also allowing for a clear separation between the train and test accuracy prior to grokking as seen in Figure 1.

To inference this model we have saved the model check point as `ckpt_2000_epochs_1_3.pt` under the `division_grokking` directory. The test set is saved under the same directory as `test_97.txt`. To inference one simply has to load the model and the tokenizer by calling the `load_model_and_tokenizer` function defined in the `inference.py` file and input the model’s checkpoint path. After loading the model and tokenizer one can inference the model by calling the `generate_output` function also defined in `inference.py`.

1.4 Analysis

1.4.1 Study 1

Study 1 focuses on the relationship between the fraction of data used for training and the required number of epochs to successfully grok.

Motivation:

While we were training the model to grok under modular division for section 1.3 we struggled to get the model to grok utilizing a training data percentage of 50% of total data. However, we quickly realized that as we decreased the fraction of data used for training the number of epochs it took for our model to grok increased. As we increased the fraction of data used for training we noticed that the model generalized much quicker which in many cases led to no grokking phase. As a result these observations prompted our investigation onto how the fraction of data used for training impacts the number of epochs needed for the model to grok.

Methods:

To accomplish this we set our model architecture and hyperparameters to those described in the grokking paper:

- Model: nanoGPT (2 layers, 4 heads, embedding size of 128)

- Block size of 16, batch size of 512
- Dropout of 0.1, bias: True
- Optimizer: AdamW, $\beta_1 = 0.9$, $\beta_2 = 0.98$.
- Learning weight of 10^{-3} , weight decay of 1.0
- Epochs: 2000 warmup: 10 steps

We then varied the training set fraction while keeping the total dataset fixed. We specifically trained and evaluated models for training set fractions of 25%, 23%, 20%, and 18%. These values were chosen as they allowed the model to successfully grok without requiring large quantities of compute.

For each configuration we measured training and test loss, training and test accuracy, and the number of epochs required to reach a test accuracy of 100%.

Results:

Training Data %	Epochs to Reach 100% Test Accuracy
25%	513
23%	884
20%	1212
18%	1781

Table 1: Epochs to reach 100% test accuracy depending on training data percent.

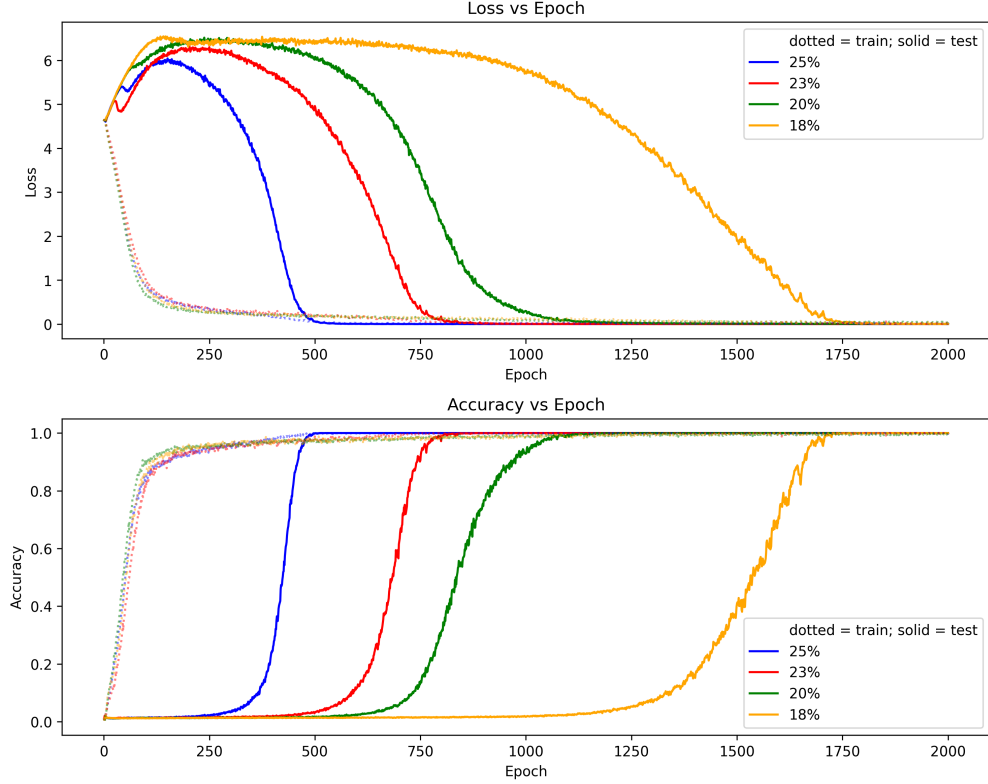


Figure 2: Effects of differing training data size on speed to grok

Our results clearly show that as the fraction of data used for training decreased the model takes longer to grok. In particular we can see that the decrease from 25% to 18% caused an the number of epochs to successfully grok to jump from 513 to 1781, an increase of 1268 epochs as seen in Table 1. Figure 1 give a visual overview comparing the training and test loss for each model thus showing how a lower fraction of data used for training causes the model to take longer to grok. Overall, our data conclusively showed that as the as the fraction of data used for training decreased the model has to train for a larger number of epochs to successfully grok.

1.4.2 Study 2

Our second study aimed to investigate how learning rate affected the convergence and stability of the model during grokking.

Motivation:

While we experimented with our model we realized that keeping all other hyperparameters and architecture constant, we could effect the rate of grokking by solely adjusting the learning rate. We also noticed that there was a clear cut off after which any learning rate higher would cause the model to not converge to a validation accuracy of 100%. As a result we wanted to investigate the effect of learning weight choice on the rate of grokking as well on the stability of the trained model.

Methods:

Similar to study 1, we set our model architecture and hyperparameters to those described in the grokking

paper while also fixing the training data percentage to 18% of total data:

- Model: nanoGPT (2 layers, 4 heads, embedding size of 128)
- Block size of 16, batch size of 512
- Dropout of 0.1, bias: True
- Optimizer: AdamW, $\beta_1 = 0.9$, $\beta_2 = 0.98$.
- weight decay of 1.0
- Epochs: 2000 warmup: 10 steps

We then varied the learning while keeping all other hyperparameters fixed. We specifically trained We then trained 7 models with learning rates of 10^{-3} , $3 \cdot 10^{-3}$, $5 \cdot 10^{-3}$, $8 \cdot 10^{-3}$, $9 \cdot 10^{-3}$, 10^{-2} , and $2 \cdot 10^{-2}$. These values were chosen as they allowed the model to successfully grok without requiring large quantities of compute and allowed us to approach the learning rate at which the model can no longer grok.

For each model configuration we measured training and test loss, training and test accuracy, and the number of epochs required to reach a test accuracy of 100%.

Results:

Learning Rate	Epochs to Reach 100% Test Accuracy
10^{-3}	1781
$3 \cdot 10^{-3}$	625
$5 \cdot 10^{-3}$	658
$8 \cdot 10^{-3}$	337
$9 \cdot 10^{-3}$	451
$1 \cdot 10^{-2}$	329
$2 \cdot 10^{-2}$	Did Not Reach 100%

Table 2: Epochs to reach 100% test accuracy depending on model learning rate.

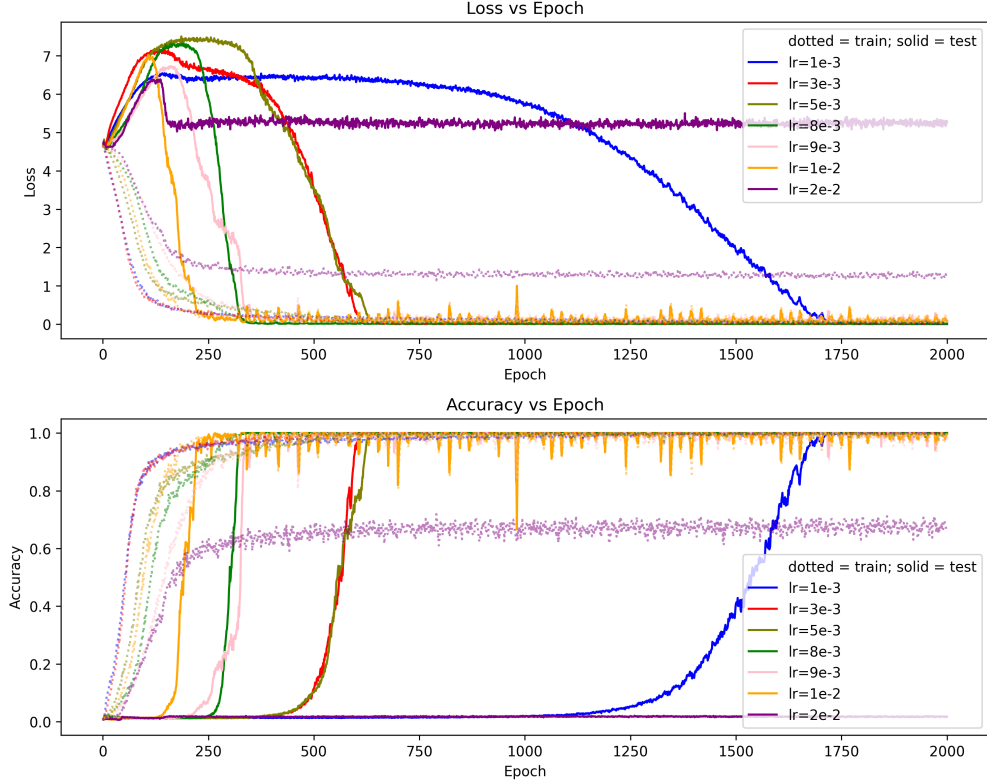


Figure 3: Effects of differing learning rate on speed to grok

Our results concluded that in general as the learning rate increased the model requires fewer epochs to grok and reach a test accuracy of 100%. While this trend holds in general we did notice that for $lr = 0.3 \cdot 10^{-3}$ and $lr = 0.5 \cdot 10^{-3}$ the accuracies and losses are nearly identical. Additionally, $lr = 0.8 \cdot 10^{-3}$ and $lr = 0.9 \cdot 10^{-3}$ match this trend as well but not as extreme as $lr = 0.3 \cdot 10^{-3}$ and $lr = 0.5 \cdot 10^{-3}$. Also, as seen in Table 2, the number of epochs required to reach 100% test accuracy showed some slightly inconsistent results as the learning rate of $0.3 \cdot 10^{-3}$ hit 100% quicker than the learning rate of $0.5 \cdot 10^{-3}$. This also held true for the learning rates $0.8 \cdot 10^{-3}$ and $0.9 \cdot 10^{-3}$. While these inconsistencies were not what we expected we saw that the general trend of decreasing the learning rate caused grokking to occur faster held overall. We also noticed that for any learning rate greater than 10^{-2} the model would not grok and would maintain a test accuracy of approximately 0 and a test loss of approximately 5.2. Additionally for when the learning rate is equal to 10^{-2} we noticed that while it would grok the quickest, it was not very stable as post grokking as training continued the train and test accuracy would regularly dip down and the train and test loss would regularly dip up as seen in Figure 3. Overall, our tests concluded that for a learning rate less than 10^{-2} our model would consistently grok and would be stable post grokking.

2 Test-Time Scaling

2.1 Warm-Up

We evaluate Qwen3-4B on AIME-2024 (30 problems) using greedy decoding ($T=0$) under two conditions, with thinking enabled and with thinking disabled. Inference is accelerated using vLLM with PagedAttention. Answers are evaluated using `exact_match`, which requires the model to produce its final answer inside a `\boxed{\}` environment.

The no-thinking condition uses a 2048-token answer budget. The thinking condition uses a 16,000-token budget (thinking and answer combined). We found that this is necessary to avoid truncating the model’s reasoning traces at the 8,000-token ceiling observed in preliminary runs.

Condition	Accuracy	Problems Solved (/ 30)
No thinking (greedy)	0.1667	5
With thinking (greedy)	0.5667	17

Table 3: Greedy decoding accuracy on AIME-2024 using `exact_match` evaluation.

Thinking improves accuracy by 40 percent (5/30 to 17/30). The no-thinking accuracy is likely an underestimate under `exact_match` because the model often produces a correct derivation but exhausts its token budget before writing the final `\boxed{}` answer.

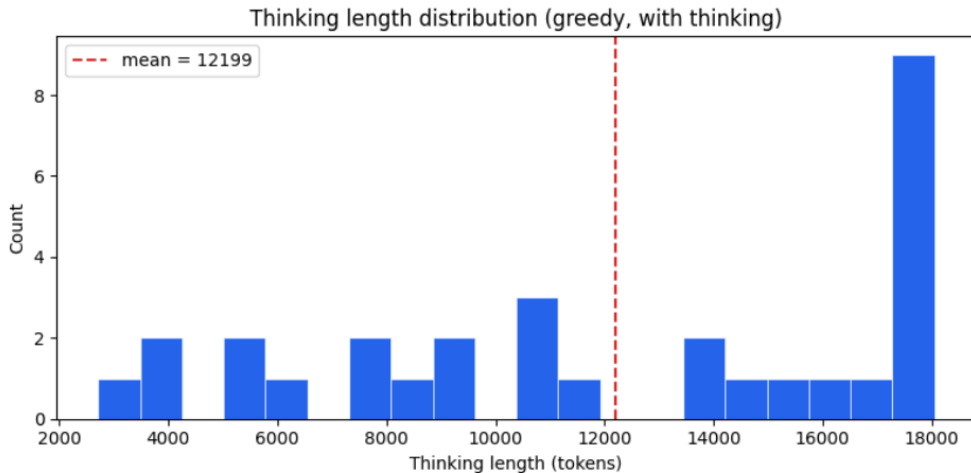


Figure 4: Distribution of thinking lengths (in tokens) under greedy decoding with thinking enabled. Mean = 12,199 tokens. The spike near 18,000 tokens indicates that a subset of problems still hit the generation ceiling even at the expanded budget.

The model’s natural reasoning length for AIME-2024 problems is 12,000-18,000 tokens. The average thinking length is 12,199 tokens, shown in 4. The distribution is roughly bimodal, with clusters likely being divided between easier and harder problems (which may require more tokens).

In addition, thinking traces are parsed by locating `<think>` and `</think>` delimiters. The answer section is extracted as all text following `</think>`.

2.2 Scaling Experiments

Sequential scaling is more compute-efficient. At equal total thinking token budgets, sequential scaling consistently outperforms parallel majority voting:

Strategy	Total thinking tokens	Exact accuracy
Sequential (budget = 16,000)	330,933	0.6333
Sequential (budget = 32,000)	414,749	0.7000
Parallel ($n = 8$, 4,000 tok/sample)	1,392,257	0.4000

Table 4: Sequential vs. parallel accuracy at comparable or lower compute budgets.

Sequential scaling achieves 0.63 accuracy using 330,933 total thinking tokens, whereas parallel scaling requires over 4x the compute to reach just 0.4. Sequential computation allows later tokens to build on earlier intermediate results, enabling more iterative reasoning. In contrast, parallel samples start from scratch independently.

However, parallel scaling is preferable when minimizing latency and when hardware is available. Each of the independent samples can be generated simultaneously across GPUs.

Access to an oracle that can judge whether an answer is "good enough" would enable **best-of-n** selection, which is an upper bound on parallel scaling. Rather than committing to the majority vote, oracle selects the first correct answer from the n samples and stops early.

In our experiments, however, **majority** and **best-of-n** were identical across values of n . This indicates that sample diversity is very low at $T=0.6$. Every problem the model solves correctly at least once, it solves on every sample. All problems it fails, it fails on every sample. There aren't problems where oracle would provide a gain over majority voting.

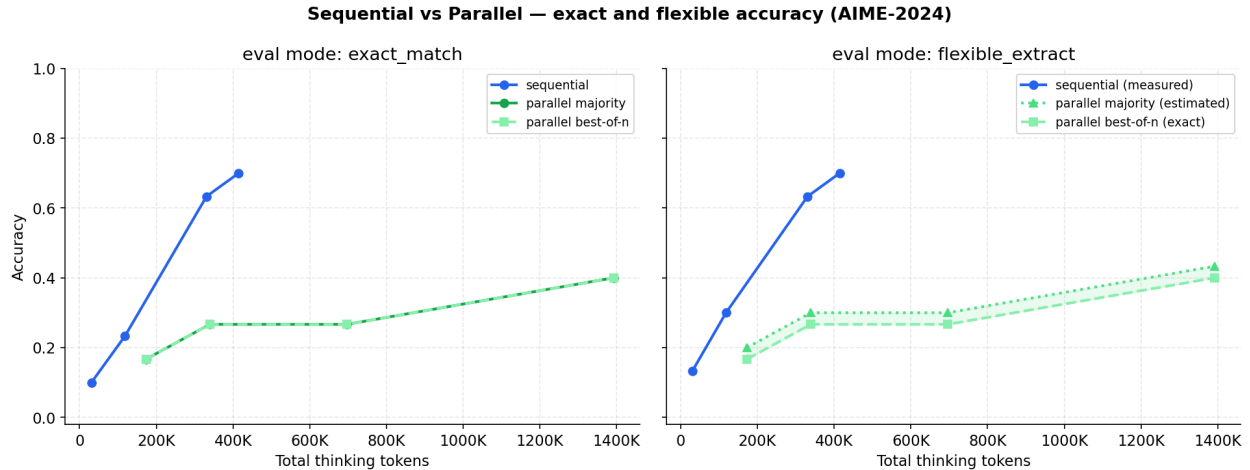


Figure 5: Comparison between sequential and parallel scaling, with two accuracies (**exact_match** and **flexible_extract**).

Flexible extract accuracy for parallel scaling is not shown separately as it was not tracked during the original run. Based on the sequential results, the gap between **exact_match** and **flexible_extract** is ≤ 0.07 at a 4,000-token per-sample budget and diminishes at larger n .

2.3 Qualitative Analysis

Problems that are unsolved by sequential scaling at low budgets but solved at higher budgets:

- Sphere containing a rectangular box (gold = 721)
- List with mode 9 and median 9 (gold = 236)

In the first problem, the trace shows the model correctly sets up the optimization problem, but is cut off before completing the Lagrange multiplier calculation. At a higher budget of 16,000 tokens, the model uses 12,191 tokens and finishes naturally. As for the second problem, there is a change in strategy. The short trace commits to the first plausible-looking solution, while the long trace exhaustively tests cases before committing.

No problems that are solved correctly at a lower budget but not at a higher budget for sequential scaling were observed in our targeted evaluation. While theoretically possible, as discussed in subsection 2.4, all four problems selected for this analysis are consistent with increasing accuracy.

Parallel (n=4, 4,000 tokens/sample) achieves majority accuracy = 0.27 vs sequential (at budget = 4,000) achieving 0.23, a marginal difference. As seen in subsection 2.4, the bottleneck is reasoning depth per sample, not diversity.

In terms of diversity of parallel traces, there are no problems where some samples are correct and others are wrong. Adding more samples to each chain provides no benefit when the model consistently fails to identify the correct approach within that budget. This motivated the hybrid strategy, which addresses diversity not by sampling more answers but by improving the quality of the reasoning context.

2.4 Improving Parallel Scaling

In part 2.2, the baseline parallel strategies showed:

- Zero diversity: majority vote accuracy equals best-of-n
- Compute inefficiency: parallel at n=8 achieves only 0.40 accuracy while sequential at budget=16,000 tokens achieves 0.63

We investigate two strategies targeting these problems directly.

2.4.1 Strategy 1: Increased sampling temperature ($T=1.0$)

The baseline uses $T=0.6$, which produces low-entropy distributions and near-deterministic outputs. We raise the temperature to $T=1.0$ while keeping all other parameters fixed. Higher temperatures increase the probability of lower-ranked tokens at each step, forcing the model to explore alternative reasoning paths and hypothetically creating the sample diversity.

Strategy	16K tokens	32K tokens
Baseline ($T = 0.6$)	0.2667	0.4000
Strategy 1 ($T = 1.0$)	0.2667	0.3667

Table 5: Strategy 1 majority vote accuracy vs. baseline.

Higher temperature produced **no improvement** at 16,000 tokens and a slight regression at 32,000 tokens (0.37 vs 0.4). Majority vote still equals best-of-n at 16K. This suggests that the diversity problem is not primarily a temperature issue. At $T=1.0$, the increased entropy also degrades individual sample quality, where the baseline benefits from more samples averaging out noise.

2.4.2 Strategy 2: Hybrid sequential-parallel scaling

Rather than spending the entire budget on independent parallel samples, we split it between depth and breadth.

- Sequential: run one chain using the WAIT technique using half the total budget
- Parallel: sample $n=4$ independent answers from the fixed thinking context at $T=0.6$

By investing in one long chain first, we give the model the reasoning depth it needs before diversifying the answer.

Strategy	16K tokens	32K tokens
Baseline ($T = 0.6$)	0.2667	0.4000
Strategy 2 (hybrid)	0.6000	$\geq 0.65^\dagger$

Table 6: Strategy 2 majority vote accuracy vs. baseline. $\dagger$ 32K hybrid run did not complete within the available compute; value estimated from sequential scaling trend.

The hybrid strategy achieves a +33 percent improvement over the baseline at the same budget, and exceeds the baseline’s best result while using half the compute. Spending 8,000 tokens on a single sequential chain allows the model to backtrack, self-correct, and commit to a viable solution path. Then, sampling 4 answers from that context introduces cheap diversity at the answer level without compromising reasoning quality.

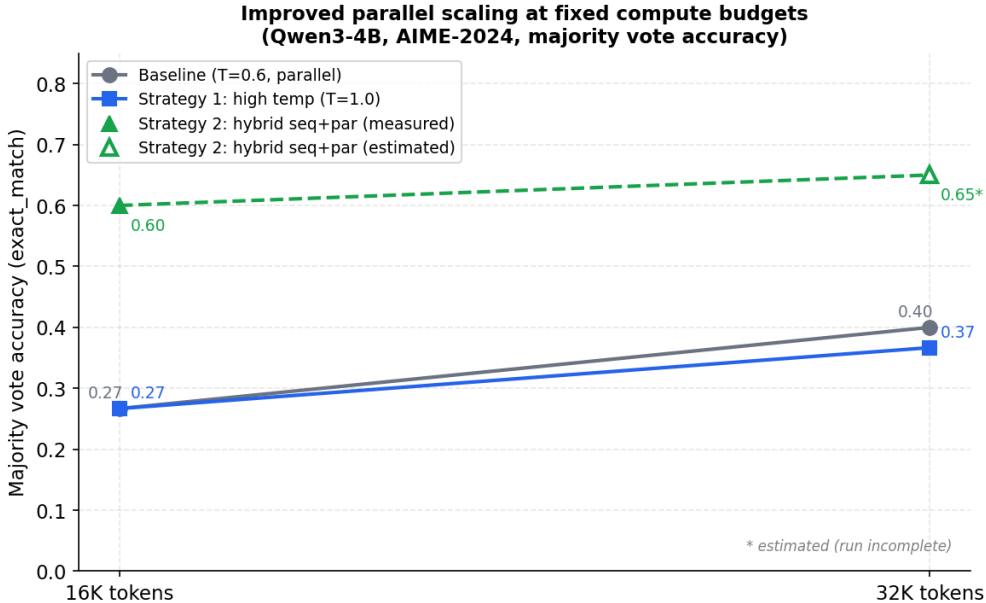


Figure 6: Comparison between improved parallel scaling strategies and baseline from part 2.2.

In summary, improving parallel scaling on hard reasoning tasks requires addressing reasoning depth, not sample diversity.

3 Synthesis and Analysis

Our experiments suggest that train-time and test-time compute help in different regimes. Extended training helps when the model has not yet internalized the rule needed to generalize, while extended inference helps when the model already has the relevant knowledge but needs more reasoning depth to execute a solution.

In Part 1, the modular arithmetic experiments were primarily generalization problems. On modular addition and subtraction, all final models achieved greater than 98% test accuracy, with delayed generalization appearing more clearly for $p = 113$ than for $p = 97$. This suggests that increased problem complexity can make grokking more visible, even when the task is still eventually learnable. The modular division experiments showed this train-time effect more sharply. With $p = 97$, reducing the training data fraction from 25% to 18% increased the number of epochs needed to reach 100% test accuracy from 513 to 1781. Similarly, increasing the learning rate generally reduced the number of epochs needed to grok, but the learning rate $2 \cdot 10^{-2}$ failed to reach 100% test accuracy. Thus, extended training is useful when additional optimization can move the model from memorizing examples toward learning the underlying algorithmic structure. However, this benefit depends heavily on having enough training signal and a stable optimization configuration.

In Part 2, the AIME-2024 experiments were primarily inference-depth problems. The pretrained Qwen3-4B model often appeared to know the relevant mathematical strategy, but short reasoning budgets prevented it from finishing the solution. Sequential scaling improved exact accuracy from 0.23 at a 4,000-token budget to 0.63 at a 16,000-token budget and 0.70 at a 32,000-token budget. In the qualitative analysis, some shorter traces correctly set up the problem but were cut off before completing the algebra, while longer traces had enough space to backtrack, test cases, and finish the calculation. This is different from grokking: the model was not learning a new rule during inference, but using more tokens to better execute a rule it already partially possessed.

The failure modes across Parts 1 and 2 show that compute is necessary but not sufficient. In Part 1, grokking was sensitive to data fraction and learning rate. Less training data delayed generalization, and too large of a learning rate prevented the model from reaching 100% test accuracy. In Part 2, longer thinking budgets helped overall, but parallel scaling was inefficient because the samples lacked diversity. Majority vote and best-of- n were identical across values of n , meaning that additional samples often repeated the same success or the same failure. Raising temperature to $T = 1.0$ also did not fix this problem; it slightly hurt accuracy at the larger budget. These failures have a common theme: more compute helps only when it changes the relevant computation. More training epochs help only if optimization can move the model toward the general rule. More inference tokens help only if they allow deeper reasoning. More parallel samples help only if they explore meaningfully different solution paths.

Given a fixed compute budget and a benchmark of easy, medium, and hard math problems, we would allocate compute based on the likely source of difficulty.

For easy problems, we would use minimal additional compute. These problems are likely already within the model’s capability and do not require long reasoning traces. In Part 2, some AIME problems were solved with greedy decoding and small budgets, so spending many extra tokens on these problems would have low marginal value. we would avoid continued finetuning and use short greedy or low-budget reasoning.

For medium problems, we would allocate most extra compute to sequential inference-time scaling. These problems appear to be within the pretrained model’s knowledge, but require enough reasoning depth to finish. This matches the AIME results, where sequential scaling was much more compute-efficient than parallel majority voting. Sequential scaling reached 0.63 exact accuracy using 330,933 thinking tokens, while parallel sampling with $n = 8$ used 1,392,257 thinking tokens but reached only 0.40 accuracy. Therefore, for medium-difficulty problems, we would prioritize one longer reasoning chain over many shallow independent samples.

For hard problems, we would use a mixture of training and inference compute. If the benchmark contains

repeated structure, such as modular arithmetic, algebraic identities, or a recurring proof pattern, we would spend some compute on continued finetuning using related tasks. This is motivated by Part 1, where modular division eventually grokked, but only after enough optimization time. After this train-time adaptation, we would still use inference-time scaling, but not plain parallel sampling. Instead, we would use the hybrid strategy from Part 2: first spend part of the budget on a sequential WAIT chain to build a strong reasoning context, then sample multiple answers from that context. This strategy achieved 0.60 accuracy at 16,000 total tokens, compared with 0.27 for baseline parallel sampling at the same budget. Overall, the best strategy is not simply to spend more compute, but to diagnose whether the problem requires learning the rule, executing the rule, or exploring multiple possible solution paths.

4 Extra Credit

We noticed two interesting experimental findings while completing this homework. One related to the increasing of training data percent causing a decrease in accuracy, and the other related to a hard cutoff for the model's learning rate for the test accuracy to converge to 100%.

When implementing section 1.2 we noticed that as we increased the training set size past 80% the model actually failed to improve training loss. This is interesting as increasing the number of samples one trains on generally reduces training loss. We further confirmed this as seen in our graphs for 1.2 we were able to get 100% test accuracy with lower training sizes while for any training set size past 80% of total data we were unable to achieve 100% test accuracy.

We also noticed that there was a hard cutoff for the learning rate to allow the test accuracy to converge to 100% as seen in section 1.4, study 2, Figure 3. We noticed that this cutoff occurred at learning rate equal 10^{-2} . While it is expected that with a large enough learning rate the model cannot converge, it is interesting that it happened at such a clean breaking point. To confirm this was the cutoff we approached with learning rate decreasing to 10^{-2} and increasing to 10^{-2} and all values of learning rate below 10^{-2} allowed the test accuracy to converge to 100% while any value greater than 10^{-2} did not allow the test accuracy to converge to 100%.